Vulnerability assessment
OpenVAS/Greenbone

Fagnani Marco, Radice Lorenzo

May 19, 2026

Contents

Chapter 1

Introduction

The experience will be focused on the vulnerability assessment of an IT system, using the OpenVAS/Greenbone tool. The main goal is to identify and evaluate potential security weaknesses in a target system.. This process involves scanning the system for known vulnerabilities, analyzing the results, and providing recommendations for remediation. The experience will also cover the configuration and usage of OpenVAS/Greenbone, as well as best practices for conducting vulnerability assessments effectively.

Chapter 2

Initial configuration

2.1 Greenbone

Greenbone/OpenVAS is an open-source vulnerability scanning tool that provides comprehensive security assessments for IT systems. To set up Greenbone/OpenVAS, firstly download the official Greenbone/OpenVAS OVA file from the Greenbone website.
https://www.greenbone.net/en/openvas-free/
Then, import the OVA file into VirtualBox 6.1 or higher. Assign at least 5GB of RAM and 2 CPU cores to the virtual machine. It's better to use about 12GB of RAM. For the first configuration, use the bridge network adapter, which allows the virtual machine to be on the same network as the host machine. Start the Greenbone/OpenVAS virtual machine and follow the on-screen instructions to complete the initial setup. Assign a user for the web interface. Once the setup is complete, access the Greenbone/OpenVAS update the feed by going to `Maintenance>Feed>Update` and wait it to synchronize (ususally hours).

2.2 Debian

Debian is an open-source operating system based on GNU/Linux. It will be used as a base to create a vulnerable machine. We used the latest stable version of Debian, which is Debian 13 Trixie.

2.2.1 Apache

Install Apache web server and create weak certificates for it. configure Apache to allow the use of weak protocols and ciphers, such as SSLv3, TLS1.1, DES, RC4 and MD5.

```
1 #!/bin/bash
2
3 apt-get update
```

```
apt-get install -y apache2 php libapache2-mod-php
rm -rf /var/lib/apt/lists/* /var/cache/apt/archives/*

a2enmod ssl
a2enmod rewrite
a2enmod headers
a2enmod proxy
a2enmod proxy_http
a2dissite 000-default.conf
echo "ServerName 127.0.0.1" | tee /etc/apache2/conf-available/servername.conf
a2enconf servername

output_dir=certs
mkdir -p $output_dir
cd $output_dir
openssl genrsa -out weak.key 1024
openssl req -new -key weak.key -out weak.csr -subj
↪  "/C=XX/ST=Lab/L=Vulnbox/O=Insecure/OU=Testing/CN=localhost"
openssl x509 -req -days 1 -in weak.csr -signkey weak.key -out weak.crt -sha1
#openssl x509 -req -days 3650 -in weak.csr -signkey weak.key -out weak_md5.crt
↪  -md5

mkdir -p /etc/apache2/ssl
cp weak.* /etc/apache2/ssl/

tee /etc/apache2/sites-available/vulnbox.conf  << EOF
<VirtualHost *:80>
    ServerName localhost

    ProxyPreserveHost On
    ProxyPass / http://127.0.0.1:8000/
    ProxyPassReverse / http://127.0.0.1:8000/

    # No security headers
</VirtualHost>

<VirtualHost *:443>
    ServerName localhost

    SSLEngine on
    SSLCertificateFile /etc/apache2/ssl/weak.crt
    SSLCertificateKeyFile /etc/apache2/ssl/weak.key

    # Allow TLS 1.0 only if OpenSSL supports it
    SSLProtocol all -SSLv3 -TLSv1.3 -TLSv1.2 -TLSv1.1
    #SSLProtocol all -SSLv3

    SSLCipherSuite ALL:NULL:EXPORT:LOW:MEDIUM:DES:RC4:MD5:@SECLEVEL=0
    #SSLCipherSuite aNULL:EXPORT:LOW:ALL:@SECLEVEL=0
    #SSLCipherSuite HIGH:!aNULL:@SECLEVEL=0
    #SSLCipherSuite ALL:NULL:EXPORT:LOW:@SECLEVEL=0

    SSLHonorCipherOrder off

    #SSLCompression on
    SSLSessionTickets On

    SSLOpenSSLConfCmd MinProtocol SSLv3
    SSLOpenSSLConfCmd CipherString DEFAULT:@SECLEVEL=0
```

```
61
62      ProxyPreserveHost On
63      ProxyPass / http://127.0.0.1:8000/
64      ProxyPassReverse / http://127.0.0.1:8000/
65
66      Header always unset Strict-Transport-Security
67      Header always unset X-Frame-Options
68      Header always unset X-Content-Type-Options
69  </VirtualHost>
70  EOF
71
72  a2ensite vulnbox.conf
73
74  tee /etc/php/7.4/apache2/php.ini << EOF
75  display_errors = On
76  display_startup_errors = On
77  allow_url_fopen = On
78  allow_url_include = On
79  disable_functions =
80  open_basedir =
81  expose_php = On
82  EOF
83
84  systemctl restart apache2
```

Any website can be used as exposed service. In our case we used an old project of ours, properly modified to be vulnerable.
`git clone -b vulnerable https://github.com/ozneroL541/artigianato-online.git`
To run it execute the following commands in the terminal:

```
1  cd artigianato-online/src
2  docker compose up --remove-orphans -d
```

2.2.2 Cups

Install the CUPS printing system and configure it to allow remote administration without authentication.

```
1  #!/bin/bash
2
3  apt-get update
4  apt-get install -y cups
5  rm -rf /var/lib/apt/lists/* /var/cache/apt/archives/*
6
7  tee /etc/cups/cupsd.conf << EOF
8  Listen 0.0.0.0:631
9
10 <Location />
11   Order allow,deny
12   Allow all
13 </Location>
14
15 <Location /admin>
16   Order allow,deny
17   Allow all
18 </Location>
19
```

```
WebInterface Yes
DefaultAuthType Basic
EOF

systemctl enable --now cups
```

2.2.3 Docker Vulnet

To simulate a complex network environment, we are using a docker container with multiple vulnerable services. The scope of this network is to show the capabilities of Greenbone/OpenVAS in scanning a complex system. To set up

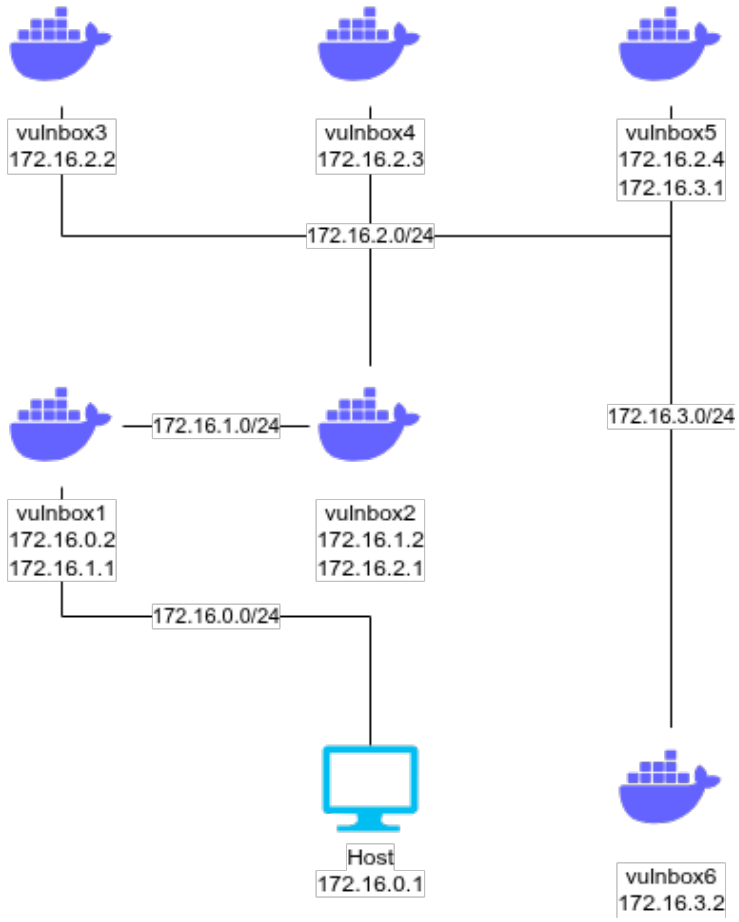

Figure 2.1: Docker network configuration

the network we used the following docker compose file:

```
#############################################################
```

```yaml
#  Vulnerability Assessment Lab
#
#  Network: 172.30.0.0/22
#    host     - 172.30.0.1/24
#    vulnbox1 - 172.30.0.2/24 172.30.1.1/24
#    vulnbox2 - 172.30.1.2/24 172.30.2.1/24
#    vulnbox3 - 172.30.2.2/24
#    vulnbox4 - 172.30.2.3/24
#    vulnbox5 - 172.30.2.4/24 172.30.3.1/24
#    vulnbox6 - 172.30.3.2/24
################################################################

networks:
  # Bridge { directly reachable from host
  net0:
    driver: bridge
    driver_opts:
      com.docker.network.bridge.name: "vulnlab0"
    ipam:
      config:
        - subnet: 172.30.0.0/24
          gateway: 172.30.0.1

  # Internal macvlan segments { isolated from host, routed only through vulnbox1
  net1:
    driver: bridge
    driver_opts:
      com.docker.network.bridge.name: "vulnlab1"
    internal: true          # blocks direct host - net1 traffic
    ipam:
      config:
        - subnet: 172.30.1.0/24
          gateway: 172.30.1.254

  net2:
    driver: bridge
    driver_opts:
      com.docker.network.bridge.name: "vulnlab2"
    internal: true          # blocks direct host - net2 traffic
    ipam:
      config:
        - subnet: 172.30.2.0/24
          gateway: 172.30.2.254

  net3:
    driver: bridge
    driver_opts:
      com.docker.network.bridge.name: "vulnlab3"
    internal: true
    ipam:
      config:
        - subnet: 172.30.3.0/24
          gateway: 172.30.3.254


services:
  # --- VULNBOX 1 { Entering node -------------------------
  vulnbox1:
    build:
```

```
      context: ./network/vulnbox-1
      dockerfile: Dockerfile
    image: vulnbox1:latest
    container_name: vulnbox1
    hostname: vulnbox1
    networks:
      net0:
        ipv4_address: 172.30.0.2
      net1:
        ipv4_address: 172.30.1.1
    cap_add:
      - NET_ADMIN
    privileged: true
    restart: unless-stopped

  # --- VULNBOX 2 { Apache Struts 2.3.31 (CVE-2017-5638) ----
  vulnbox2:
    build:
      context: ./network/vulnbox-2
      dockerfile: Dockerfile
    image: vulnbox2:latest
    container_name: vulnbox2
    hostname: vulnbox2
    networks:
      net1:
        ipv4_address: 172.30.1.2
      net2:
        ipv4_address: 172.30.2.1
    cap_add:
      - NET_ADMIN
    privileged: true
    restart: unless-stopped

  # --- VULNBOX 3 { DVWA -----------------------------------
  vulnbox3:
    build:
      context: ./network/vulnbox-3
      dockerfile: Dockerfile
    image: vulnbox3:latest
    container_name: vulnbox3
    hostname: vulnbox3
    networks:
      net2:
        ipv4_address: 172.30.2.2
    cap_add:
      - NET_ADMIN
    privileged: true
    restart: unless-stopped

  # --- VULNBOX 4 { SSH weak credentials -------------------
  vulnbox4:
    build:
      context: ./network/vulnbox-4
      dockerfile: Dockerfile
    image: vulnbox4:latest
    container_name: vulnbox4
    hostname: vulnbox4
    networks:
      net2:
```

```
120           ipv4_address: 172.30.2.3
121       cap_add:
122         - NET_ADMIN
123       privileged: true
124       restart: unless-stopped
125
126     # --- VULNBOX 5 { Samba ----------------------------------
127     vulnbox5:
128       build:
129         context: ./network/vulnbox-5
130         dockerfile: Dockerfile
131       image: vulnbox5:latest
132       container_name: vulnbox5
133       hostname: vulnbox5
134       networks:
135         net2:
136           ipv4_address: 172.30.2.4
137         net3:
138           ipv4_address: 172.30.3.1
139       cap_add:
140         - NET_ADMIN
141       privileged: true
142       restart: unless-stopped
143
144     # --- VULNBOX 6 { MySQL no root password ------------------
145     vulnbox6:
146       build:
147         context: ./network/vulnbox-6
148         dockerfile: Dockerfile
149       image: vulnbox6:latest
150       container_name: vulnbox6
151       hostname: vulnbox6
152       networks:
153         net3:
154           ipv4_address: 172.30.3.2
155       cap_add:
156         - NET_ADMIN
157       privileged: true
158       restart: unless-stopped
```

To ensure proper network setup it was crucial to set the correct routes throgh all the containers, so that they could communicate with each other in the way we wanted. To do so we wrote the following script:

```
1  #!/bin/bash
2  # Stop and remove any existing containers
3  docker compose down --remove-orphans
4  # Start the vulnbox network
5  docker compose up --remove-orphans -d
6  # Allow forwarding between containers
7  iptables -I FORWARD 1 -s 172.30.0.0/22 -d 172.30.0.0/22 -j ACCEPT
8  # Remove the default IP addresses from the bridge interfaces
9  ip addr del 172.30.1.254/24 dev vulnlab1
10 ip addr del 172.30.2.254/24 dev vulnlab2
11 ip addr del 172.30.3.254/24 dev vulnlab3
12 # Host
13 # Tell your physical machine to send lab traffic to vulnbox1
14 ip route replace 172.30.1.0/24 via 172.30.0.2
15 ip route replace 172.30.2.0/24 via 172.30.0.2
```

```
ip route replace 172.30.3.0/24 via 172.30.0.2
# vulnbox1
docker exec -u 0 vulnbox1 sysctl -w net.ipv4.ip_forward=1
docker exec -u 0 vulnbox1 ip route replace 172.30.2.0/24 via 172.30.1.2
docker exec -u 0 vulnbox1 ip route replace 172.30.3.0/24 via 172.30.1.2
docker exec -u 0 vulnbox1 ip route add default via 172.30.0.1
# vulnbox2
docker exec -u 0 vulnbox2 sysctl -w net.ipv4.ip_forward=1
docker exec -u 0 vulnbox2 ip route replace 172.30.3.0/24 via 172.30.2.4
docker exec -u 0 vulnbox2 ip route add default via 172.30.1.1
# vulnbox3
docker exec -u 0 vulnbox3 ip route add 172.30.3.0/24 via 172.30.2.4
docker exec -u 0 vulnbox3 ip route add default via 172.30.2.1
# vulnbox4
docker exec -u 0 vulnbox4 ip route replace 172.30.3.0/24 via 172.30.2.4
docker exec -u 0 vulnbox4 ip route add default via 172.30.2.1
# vulnbox5
docker exec -u 0 vulnbox5 sysctl -w net.ipv4.ip_forward=1
docker exec -u 0 vulnbox5 ip route add default via 172.30.2.1
# vulnbox6
docker exec -u 0 vulnbox6 ip route add default via 172.30.3.1
```

To let Greenbone/OpenVAS scan the containers, we had to crete further rules. Since the VM is inteded as a vulnerable machine, we had no intention to let the firewall block any traffic, therefore, firstly, we flushed all its rules and then we build up our own ruleset, which allows all the traffic from the Greenbone/OpenVAS to the container's network.

```
#!/bin/bash

# DO NOT RUN THIS ON YOUR HOST MACHINE
# VM ONLY

docker_interface="vulnlab0"
docker_subnet="172.30.0.0/22"
interface=$(ip -o -f inet addr show | awk '/scope global/ {print $2}' | head -n
↪  1)

./setup.sh
# Flush everything and start clean
iptables -F FORWARD
iptables -t nat -F POSTROUTING
nft flush chain ip raw PREROUTING

# Allow forwarding between containers
iptables -I FORWARD 1 -s $docker_subnet -d $docker_subnet -j ACCEPT

# Re-add Docker's default NAT rules
iptables -t nat -A POSTROUTING -s $docker_subnet ! -o $docker_interface -j
↪  MASQUERADE
iptables -t nat -A POSTROUTING -s 172.17.0.0/16 ! -o docker0 -j MASQUERADE

# Allow forwarding between eth and bridge
iptables -I FORWARD 1 -i $interface -o $docker_interface -j ACCEPT
iptables -I FORWARD 1 -i $docker_interface -o $interface -j ACCEPT

# Allow Docker chain
iptables -I DOCKER 1 -i $interface -d $docker_subnet -j ACCEPT
```

vulnbox1

The first vulnbox was intentended to be vulnerable to log4Shell (CVE-2021-44228), a critical vulnerability in the Apache Log4j library. However, since the container was handmade, Greenbone/OpenVAS was not able to detect it.

```
# Vulnbox 1 - Log4Shell
# Pivot node for networking
FROM maven:3.9.9-eclipse-temurin-8 AS build

WORKDIR /build
COPY pom.xml .
RUN mvn -q -e -DskipTests
↪  org.apache.maven.plugins:maven-dependency-plugin:3.3.0:go-offline

COPY src ./src
RUN mvn -q -DskipTests package

# ---- runtime ----
FROM eclipse-temurin:8-jre-jammy

# install tools for your lab
RUN apt-get update -o Acquire::AllowInsecureRepositories=true \
    -o Acquire::AllowDowngradeToInsecureRepositories=true && \
    apt-get install -y --allow-unauthenticated iproute2 curl && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/* /var/cache/apt/archives/*

WORKDIR /app
COPY --from=build /build/target/vulnapp-1.0-jar-with-dependencies.jar .

EXPOSE 8080

CMD ["java", "-jar", "vulnapp-1.0-jar-with-dependencies.jar"]
```

Even it the vulnerability was not detected, the main role of the first vulnbox was to route all the traffic to the docker network and vice versa and, in that sense, it was successful.

vulnbox2

The second vulnbox present a vulnerable version of the Apache Struts framework, which is affected by the CVE-2017-5638 vulnerability.

```
# Apache Struts 2.3.31 { CVE-2017-5638
# The Content-Type header is parsed by the Jakarta Multipart parser.
# A malformed value triggers OGNL expression evaluation → unauthenticated RCE.
# This is the exact vulnerability used in the 2017 Equifax breach.

FROM ubuntu:20.04

ENV DEBIAN_FRONTEND=noninteractive

# Install Java 8 + Tomcat 8 + wget
RUN apt-get update -qq && \
    apt-get install -y -qq \
        openjdk-8-jdk-headless \
```

```
14          wget \
15          curl \
16          unzip \
17          iproute2 && \
18      rm -rf /var/lib/apt/lists/*
19
20  ENV JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
21  ENV PATH="$JAVA_HOME/bin:$PATH"
22
23  # -- Tomcat 8.5 -------------------------------------------
24  ENV TOMCAT_VERSION=8.5.87
25  ENV CATALINA_HOME=/opt/tomcat
26
27  RUN wget -q
    ↪  "https://archive.apache.org/dist/tomcat/tomcat-8/v${TOMCAT_VERSION}/bin/apache-tomcat-${TOMCAT_VERSION}.tar.g
    ↪  \
28          -O /tmp/tomcat.tar.gz && \
29      mkdir -p "$CATALINA_HOME" && \
30      tar xzf /tmp/tomcat.tar.gz -C "$CATALINA_HOME" --strip-components=1 && \
31      rm /tmp/tomcat.tar.gz
32
33  # -- Struts 2.3.31 showcase WAR (contains the vulnerable REST plugin) --
34  ENV STRUTS_VERSION=2.3.31
35
36  RUN wget -q
    ↪  "https://archive.apache.org/dist/struts/${STRUTS_VERSION}/struts-${STRUTS_VERSION}-apps.zip"
    ↪  \
37          -O /tmp/struts-apps.zip && \
38      unzip -q /tmp/struts-apps.zip
        ↪  "struts-${STRUTS_VERSION}/apps/struts2-showcase.war" \
39          -d /tmp/struts-extract/ && \
40      cp "/tmp/struts-extract/struts-${STRUTS_VERSION}/apps/struts2-showcase.war" \
41          "${CATALINA_HOME}/webapps/struts2-showcase.war" && \
42      rm -rf /tmp/struts-apps.zip /tmp/struts-extract
43
44  # Remove default Tomcat apps to keep the attack surface clean/focused
45  RUN rm -rf \
46      "$CATALINA_HOME/webapps/ROOT" \
47      "$CATALINA_HOME/webapps/docs" \
48      "$CATALINA_HOME/webapps/examples" \
49      "$CATALINA_HOME/webapps/host-manager" \
50      "$CATALINA_HOME/webapps/manager"
51
52  # Tomcat manager creds (useful for post-exploit exploration)
53  COPY tomcat-users.xml "$CATALINA_HOME/conf/tomcat-users.xml"
54
55  EXPOSE 8080
56
57
58  CMD ["sh", "-c", "$CATALINA_HOME/bin/catalina.sh run"]
```

vulnbox3

The third vulnbox is based on DVWA (Damn Vulnerable Web Application), a web application that is intentionally vulnerable to various types of attacks. We had no clue if Greenbone/OpenVAS would have been able to detect any vulnerabilities here, but since it's in fact the scope of such experiment, we

decided to include it.

```
# Vulnbox 3 { DVWA
# Damn Vulnerable Web Application (DVWA) is a PHP/MySQL web app designed for security training.
# It contains multiple vulnerabilities of varying difficulty, making it ideal for practice.

FROM vulnerables/web-dvwa

ENV MYSQL_PASS=p@ssw0rd
ENV HOSTNAME=vulnbox3


EXPOSE 80 443
```

vulnbox4

The fourth vulnbox is a simple Ubuntu 22.04 image with weak SSH credentials.

```
# Vulnbox 4 - SSH Service with Weak Credentials
# This container runs an SSH server with weak credentials for testing purposes.
FROM ubuntu:22.04

RUN apt-get update -qq && \
    apt-get install -y -qq openssh-server iproute2 && \
    rm -rf /var/lib/apt/lists/* && \
    useradd -m -s /bin/bash admin; \
    echo 'admin:admin123' | chpasswd; \
    useradd -m -s /bin/bash user; \
    echo 'user:password' | chpasswd; \
    mkdir -p /run/sshd && \
    sed -i 's/#PermitRootLogin.*/PermitRootLogin yes/' /etc/ssh/sshd_config && \
    sed -i 's/#PasswordAuthentication.*/PasswordAuthentication yes/' /etc/ssh/sshd_config && \
    echo 'root:toor' | chpasswd

EXPOSE 22


CMD ["/usr/sbin/sshd", "-D"]
```

vulnbox5

The fifth vulnbox is a weak Samba server.

```
# Vulnbox 5 - Samba (EthernalBlue-era misconfiguration)
# This container runs a Samba server with a misconfiguration that allows for potential exploitation.

FROM dperson/samba

RUN mkdir -p /public /private && \
    chmod 777 /public && \
    chmod 700 /private

ENV USERID=1000 \
    GROUPID=1000
```

```
12
13
14  CMD ["-u", "guest;guest", \
15       "-s", "public;/public;yes;no;yes;all;none", \
16       "-s", "private;/private;no;no;no;all;none"]
```

vulnbox6

The sixth vulnbox is a MySQL server with no password.

```
1  FROM mysql:5.7
2
3  ENV MYSQL_ALLOW_EMPTY_PASSWORD=yes
4
5  RUN yum clean all && \
6      yum install -y iproute && \
7      yum clean all && \
8      rm -rf /var/cache/yum
```

2.2.4 FTP

The FTP server is a simple instance of `vsftpd` with multiple weak configurations, such as:

```
1  local_enable=YES
2  write_enable=YES
3  pam_service_name=vsftpd
```

2.2.5 Samba

The Samba server is configured to allow insecure access to a shared folder.

```
1  #!/bin/bash
2
3  apt-get install -y samba
4
5  tee /etc/samba/smb.conf << EOF
6  [global]
7     workgroup = WORKGROUP
8     server string = Vuln Samba
9     map to guest = Bad User
10    guest account = nobody
11
12    # Weak / legacy protocol support
13    server min protocol = NT1
14    ntlm auth = yes
15    lanman auth = yes
16
17    # Disable signing
18    server signing = disabled
19
20 [vulnshare]
21    path = /srv/samba/vuln
22    browsable = yes
23    writable = yes
24    guest ok = yes
```

```
   read only = no
   force user = nobody
EOF

mkdir -p /srv/samba/vuln
chmod 777 /srv/samba/vuln
systemctl enable --now smbd
```

2.2.6 SSH

The SSH server is configured with weak configurations, such as:

```
PermitRootLogin yes
PasswordAuthentication no
```

2.2.7 Syslog

The Syslog server is configured to allow remote logging without authentication on port 514.

```
#!/bin/bash

#apt-get update
#apt-get install -y rsyslog
#apt-get clean
#rm -rf /var/lib/apt/lists/*

# Enable UDP + TCP syslog reception
cat << EOF > /etc/rsyslog.d/10-remote.conf
# Load modules
module(load="imudp")
input(type="imudp" port="514")

module(load="imtcp")
input(type="imtcp" port="514")

# Log EVERYTHING from remote hosts
*.* /var/log/remote.log

# No rate limiting (intentionally bad)
\$SystemLogRateLimitInterval 0
EOF

# Ensure rsyslog is enabled
systemctl enable rsyslog
systemctl restart rsyslog

mkdir -p /var/log/
chmod 777 -R /var/log/
```

2.3 Windows XP

Windows XP is an old operating system that is no longer supported by Microsoft, which makes it vulnerable to various types of attacks. Just it's mere presence in the network is a vulnerability, since it can be easily exploited by

attackers. To set up Windows XP, we just installed its virtual machine on VirtualBox and connected it to the same network of the other machines.

2.4 Network configuration

The whole network configuration is a simple virtual network created with VirtualBox, which allows all the machines to communicate with each other and have no access to other networks, such as the Internet.

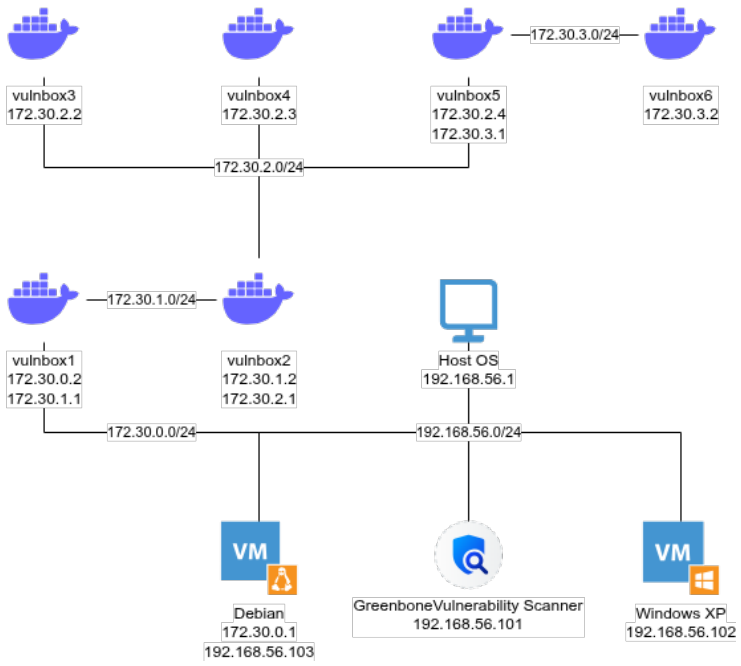

Figure 2.2: Full laboratory's network